Qwen

# От **PyTorch** к **Zen 2:** Стратегия создания сверхпроизводительного фреймворка **Reto-Origen** для **CPU-only** обучения

## Анализ Целевой Аппаратной Платформы: Архитектура **Zen 2** и Ее Импликации для Оптимизации

Для достижения цели по созданию фреймворка Reto-Origen, который превосходит существующие решения по всем ключевым параметрам при обучении моделей исключительно на центральном процессоре, первоочередной задачей является глубокий анализ целевой аппаратной конфигурации. Исследование сосредоточено на процессоре AMD Ryzen 5 3600 и системе с 16 ГБ оперативной памяти. Понимание микроархитектурных особенностей, тактовых частот, иерархии кэша и возможностей набора инструкций является фундаментом для любой последующей инженерной оптимизации. Этот раздел предоставляет исчерпывающий технический разбор процессора Ryzen 5 3600 на основе архитектуры Zen 2, выявляя как возможности, так и ограничения, которые будут определять архитектуру и поведение будущего фреймворка.

Процессор AMD Ryzen 5 3600, входящий в линейку потребительских решений, основан на микросхеме "Matisse", которая использует архитектуру Zen 2 [9] [33]. Это многочиповая модульная (MCM) конструкция, где несколько кристаллов с ядрами (Core Complex Dies, CCDs) соединены с отдельным I/O Die (IOD) [9]. Для Ryzen 5 3600 это означает наличие одного CCD. Этот CCD содержит шесть ядер процессора, каждое из которых поддерживает два потока выполнения благодаря технологии Simultaneous Multithreading (SMT), что в сумме дает 12 физических потоков для одновременной обработки задач [5] [180]. Такая конфигурация определяет теоретический предел параллелизма, доступный для фреймворка Reto-Origen. Любая успешная стратегия распараллеливания вычислений должна быть способна эффективно загрузить все 12 потоков, например, с помощью стандартов OpenMP или TBB. Однако простое увеличение количества потоков не гарантирует линейного роста производительности;

ключевую роль играет управление взаимодействием между ядрами и их кэшами.

Тактовые частоты Ryzen 5 3600 составляют базовую частоту 3.6 ГГц и максимальную частоту Turbo Boost до 4.05 ГГц [9] [31]. Высокая базовая частота является важным преимуществом для задач машинного обучения, где время ответа на один шаг обучения или один вывод имеет большое значение. Скорость работы цикла (clock cycle time) напрямую влияет на производительность задач, чувствительных к задержкам, таких как итерации в цикле обучения. Возможность стабильно работать на высоких частотах без перегрева будет зависеть от эффективности системы охлаждения и настроек BIOS, но сам факт наличия высокого потолка частоты — это ценное свойство процессора.

Одним из самых значительных преимуществ Ryzen 5 3600 является его иерархия кэширования. На уровне L1 у каждого ядра есть собственный кэш данных размером 32 КБ и кэш инструкций размером 32 КБ [9] [11]. Эти кэши являются быстрейшей формой памяти процессора. Затем следует уровень L2 кэша, также принадлежащий каждому ядру, размером 512 КБ [9] [11]. Наконец, на уровне L3 находится общий кэш объемом 32 МБ, доступный всем шести ядрам [5] [9]. Объем L3 кэша в 32 МБ является огромным преимуществом для потребительского процессора и имеет решающее значение для производительности на CPU. Большинство небольших и средних моделей машинного обучения (веса, активации, градиенты) могут поместиться полностью в этом кэше. Когда данные находятся в L3 кэше, доступ к ним происходит практически мгновенно по сравнению с обращением к системной оперативной памяти (DRAM). Таким образом, эффективное управление данными таким образом, чтобы максимизировать их пребывание в L3 кэше, станет главным приоритетом для Reto-Origen. Это достигается за счет оптимизации локальности кэша (cache locality) в коде операций и правильного планирования вычислений.

Важнейшим аспектом для оптимизации является набор инструкций, поддерживаемый процессором. Ryzen 5 3600 поддерживает инструкции AVX (Advanced Vector Extensions) и, что более важно, AVX2 [4] [5]. AVX2 позволяет выполнять векторные операции над 256-битными регистрами, что критически важно для ускорения линейно-алгебраических вычислений, составляющих основу большинства нейронных сетей. В отличие от некоторых более ранних архитектур, на Zen 2 инструкции AVX2 выполняются за один такт, в то время как

у более ранних архитектур Zen требовалось два такта [9] [120]. Это обеспечивает высокую производительность для операций, ограниченных пропускной способностью вычислений, таких как матричное умножение и свертки. Кроме того, каждый кристалл Zen 2 имеет две 256-битные FMA (умножение-сложение) единицы на ядро, что позволяет одновременно выполнять две 256-битные операции [10] [11]. Это означает, что для достижения максимальной производительности в вычислительно-интенсивных ядрах необходимо, чтобы компилятор генерировал код, эффективно использующий обе FMA единицы. Это делает использование современных компиляторов с продвинутыми механизмами векторизации абсолютно необходимым.

Хотя Ryzen 5 3600 является односокетным процессором, его MCM-архитектура создает внутреннюю структуру, аналогичную NUMA (Non-Uniform Memory Access). Два Core Complex Dies (CCD) имеют доступ к разным половинам L3 кэша [9]. Хотя для Ryzen 5 3600 используется только один CCD, понимание этой архитектуры полезно для масштабирования и для понимания принципов распределения нагрузки. Для Reto-Origen это означает, что при планировании потоков следует стремиться к тому, чтобы потоки, имеющие тесную связь друг с другом (например, потоки, обрабатывающие соседние части одной большой матрицы), были назначены на ядра, принадлежащие одному CCX (Core Complex), чтобы максимизировать совместное использование L3 кэша.

Система с 16 ГБ оперативной памяти представляет собой наиболее серьезное ограничение. Современные модели, особенно большие языковые модели (LLMs), требуют десятков, если не сотен гигабайт ОЗУ для обучения. Например, в одном из исследований использовалась система с 32 ГБ RAM для экспериментов на Ryzen 5 3600 [158] [159], а в другом — 24 ГБ [145]. Это указывает на то, что даже в исследовательских кругах 16 ГБ считается минимальным порогом, часто недостаточным для крупных моделей. Следовательно, Reto-Origen должен быть спроектирован с учетом жесткой нехватки памяти. Это требует внедрения продвинутых техник управления памятью, таких как: 1. Квантование: Снижение числовой точности весов и активаций (например, с 32-битных чисел с плавающей запятой (FP32) до 8-битных целых чисел (INT8)). Это позволяет уменьшить требования к памяти примерно в 4 раза и может значительно ускорить вычисления на процессорах, поддерживающих такие операции. 2. Спарсификация: Использование разреженных представлений для весов, когда большинство значений равны нулю. 3. **Gradient Checkpointing:** Техника, которая жертвует временем вычислений ради экономии памяти, пересчитывая некоторые промежуточные значения во время обратного прохода

вместо их сохранения. 4. Эффективное управление выделением памяти: Использование специализированных менеджеров памяти, которые минимизируют фрагментацию и ускоряют выделение/освобождение блоков памяти.

Скорость и тип оперативной памяти также играют важную роль. Предполагается, что система использует DDR4. Источники показывают, что скорость памяти напрямую влияет на производительность Ryzen [31] [183]. Например, переход с 2133 МГц до 3200 МГц может дать заметный прирост в бенчмарках [183]. Стандартная частота памяти для платформы Ryzen 3000 — 2666 МГц [31]. Латентность памяти составляет около 103 наносекунд [31]. Reto-Origen должен учитывать эти характеристики при проектировании алгоритмов, которые могут быть ограничены пропускной способностью памяти.

| Характеристика | Значение для **Ryzen 5 3600** | Импликации для **Reto-Origen** |
|---|---|---|
| Микроархитектура | Zen 2 [9] | Необходимость использования инструкций и оптимизаций, специфичных для Zen 2 (AVX2 за 1 такт, 2 FMA на ядро). |
| Ядра / Потоки | 6 / 12 [5] [180] | Теоретический предел параллелизма. Фреймворк должен эффективно использовать 12 потоков через OpenMP или аналоги. |
| Базовая / **Turbo** Частота | 3.6 ГГц / до 4.05 ГГц [9] [31] | Высокая базовая частота важна для задач с низкой задержкой. Производительность зависит от термического дизайна. |
| **L1** Кэш | 32 КБ D + 32 КБ I на ядро [9] [11] | Крайне быстрая память. Оптимизация локальности кэша на этом уровне (data locality) критична. |
| **L2** Кэш | 512 КБ на ядро [9] [11] | Быстрая память, служащая буфером между L1 и L3. Эффективное использование помогает снижать давление на L3. |
| **L3** Кэш | 32 МБ [5] [9] | Ключевой фактор производительности на CPU. Данные модели должны максимально долго оставаться в кэше. |
| Поддерживаемые инструкции | AVX, AVX2 [4] [5] | AVX2 (256-bit SIMD) необходим для векторизации линейной алгебры. Инструкции Zen 2 выполняются за 1 такт. |
| Количество **FMA** на ядро | 2 (256-bit) [10] [11] | Компилятор должен генерировать код, использующий обе FMA единицы для максимальной производительности. |
| Объем ОЗУ | 16 ГБ [143] [160] | Главное ограничение. Требуется встроенная поддержка квантования, gradient checkpointing и других техник экономии памяти. |
| Латентность ОЗУ | ~103 нс [31] | Влияет на общую производительность. Алгоритмы, чувствительные к задержкам памяти, должны быть оптимизированы. |

В заключение анализа целевой платформы, можно сделать вывод, что AMD Ryzen 5 3600 представляет собой мощный, но узкоспециализированный "сервер" для вычислений на центральном процессоре. Его сильные стороны — это огромный кэш L3 и высокая производительность в векторных вычислениях

благодаря AVX2. Основная проблема — это ограниченный объем оперативной памяти в 16 ГБ. Следовательно, стратегия оптимизации Reto-Origen должна быть сфокусирована на трех столпах: 1) создание кода, который максимально эффективно использует кэш-иерархию Zen 2; 2) генерация машинного кода, который полностью раскрывает потенциал 256-битных инструкций AVX2 и двух FMA-устройств на ядро; 3) внедрение продвинутых механизмов управления памятью, позволяющих преодолеть ограничения в 16 ГБ ОЗУ. Любая неэффективная реализация в этих областях немедленно приведет к значительному падению производительности или невозможности обучения даже относительно небольших моделей.

## Стратегические Подходы к Компиляции и Связыванию для Максимальной Производительности

После детального анализа аппаратной платформы следующим логическим шагом является выбор и настройка инструментария компиляции, который является краеугольным камнем для достижения максимальной производительности. Программный код, сгенерированный компилятором, определяет, насколько хорошо он сможет использовать уникальные особенности процессора AMD Ryzen 5 3600 (архитектура Zen 2). Выбор неверного компилятора или некорректная настройка его флагов могут привести к тому, что даже идеально написанный алгоритм будет выполняться значительно медленнее, чем мог бы. Для Reto-Origen цель состоит не просто в компиляции кода, а в генерации максимально эффективного машинного кода, специально адаптированного под Zen 2. Этот раздел подробно рассматривает стратегии компиляции, уделяя особое внимание компилятору AOCC (AMD Optimizing C/C++ Compiler) и передовым техникам оптимизации, таким как Profile-Guided Optimization (PGO) и Link-Time Optimization (LTO).

Основным и наиболее мощным инструментом для данной задачи является AMD Optimizing C/C++ Compiler (AOCC). AOCC — это не просто сборка LLVM/Clang, а версия, глубоко интегрированная с экосистемой AMD и специально оптимизированная для процессоров на архитектуре "Zen" [37] [162212]. Он основан на LLVM и предназначен для генерации высокопроизводительного машинного кода для Linux [37]. Использование AOCC вместо стандартных компиляторов, таких как GCC или базовый Clang, является первым и самым важным шагом на

пути к созданию сверхбыстрого фреймворка. AOCC предлагает ряд агрессивных уровней оптимизации, таких как `-Ofast`, который включает в себя больше оптимизаций, чем стандартный `-O3`, и может нарушать некоторые строгие стандарты языка в пользу скорости [42] [46].

Ключевым элементом настройки AOCC является флаг `-march=ARCH`. Этот флаг указывает компилятору, для какой конкретной архитектуры генерировать код, что позволяет ему использовать все доступные инструкции набора инструкций. Для процессора Ryzen 5 3600, который основан на Zen 2, необходимо использовать флаг `-march=znver2` [10] [39]. Без этого флага компилятор может по умолчанию генерировать код для более старой архитектуры (например, для Zen 1, что было стандартом в старых версиях AOCC [39]), что приведет к пропуску всех преимуществ Zen 2, включая двойную ширину векторных единиц и однотактное выполнение AVX2 [9] [120]. Использование `-march=native` также является вариантом, так как он автоматически определяет архитектуру текущей машины, но явное указание `-march=znver2` предпочтительнее для воспроизводимости сборок [43].

Помимо базовой настройки архитектуры, существует ряд специфических флагов, которые помогают направить усилия компилятора в нужное русло. Для вычислительно-интенсивных ядер, таких как матричное умножение, важно, чтобы компилятор эффективно использовал все доступные исполнительные блоки. Zen 2 имеет два 256-битных FMA устройства на ядро [10]. Чтобы гарантировать, что компилятор попытается использовать оба, рекомендуется использовать флаги `-mprefer-vector-width=256` и `-mfma` [10]. Также полезно выравнивать циклы по 32-байтным границам с помощью `-falign-loops=32` и `-falign-functions=32`, что соответствует способу, которым Zen 2 загружает данные в кэш, и помогает избежать штрафов за несовпадение адресов [10].

Профилирование на основе использования — это еще одна мощная техника, которая может дать значительный прирост производительности. PGO работает в два этапа. На первом этапе код компилируется с флагом `-fprofile-generate`, что добавляет в него инструментарий для сбора информации о том, какие ветви исполняются чаще всего, и каков порядок вызова функций. Затем программа запускается на репрезентативном наборе рабочих нагрузок (например, проходит один полный цикл обучения на реальной модели). После завершения программа генерирует файлы профилей. На втором этапе код компилируется заново, но уже с флагом `-fprofile-use=path/to/profiles`, и компилятор использует собранную информацию для оптимизации. Например, он

может расположить более вероятные ветви условных переходов рядом друг с другом, чтобы улучшить предсказание ветвлений, или оптимизировать расположение кода в памяти, чтобы уменьшить трешинг $\mu$OP-кэша [10]. Для систем Zen 2 PGO может обеспечить прирост производительности от 15% до 25%, что является огромным результатом за счет относительно простой процедуры [10].

Связывание на этапе компиляции (Link-Time Optimization, LTO) — это еще одна обязательная для использования оптимизация. LTO позволяет компилятору видеть весь проект целиком, а не по отдельным файлам. Это открывает возможность для более глубоких оптимизаций, таких как удаление неиспользуемого кода, перемещение кода между функциями и слияние функций. Для AOCC рекомендуется использовать флаг `-flto=thin` для более быстрой компиляции или `-flto=full` для максимальной оптимизации, хотя последний может значительно увеличить время сборки [10]. Важно использовать современный линкер `lld` (часть LLVM-toolchain), который лучше всего поддерживает LTO, указав флаг `-fuse-ld=lld` [10] [42].

Ниже представлена таблица, обобщающая ключевые флаги компиляции для AOCC, которые должны быть использованы при сборке Reto-Origen.

| Флаг компилятора | Назначение | Обоснование для **Ryzen 5 3600 (Zen 2)** |
|---|---|---|
| `-march=znver2` | Указывает архитектуру назначения. | Обязательно для использования всех инструкций Zen 2, включая однотактный AVX2 [10] [39]. |
| `-Ofast` | Включает агрессивные оптимизации, нарушающие стандарты. | Ускоряет вычисления за счет менее точных, но быстрых математических операций (`ffast-math`) [42] [46]. |
| `-flto=thin` | Включает LTO для межфайловых оптимизаций. | Позволяет компилятору видеть весь проект, удалять неиспользуемый код и выполнять более глубокие оптимизации [10] [42]. |
| `-fuse-ld=lld` | Использует `lld` в качестве внешнего линкера. | `lld` обеспечивает лучшую поддержку LTO по сравнению со стандартным `ld.bfd` или `ld.gold` [10] [42]. |
| `-fprofile-generate` / `-fprofile-use` | Включает PGO. | Повышает производительность на 15-25% путем оптимизации ветвлений и расположения кода на основе реальных рабочих нагрузок [10]. |
| `-mprefer-vector-width=256` | Предпочитает векторы шириной 256 бит. | Заставляет компилятор использовать 256-битные AVX2 инструкции, которые подходят для Zen 2 [10]. |
| `-mfma` | Включает поддержку FMA-инструкций. | Необходимо для использования двух FMA-устройств на ядро Zen 2 [10]. |
| `-falign-loops=32` | Выравнивает начало циклов по 32-байтным границам. | Соответствует способу загрузки данных в кэш Zen 2, снижая накладные расходы на несовпадение адресов [10]. |

Стоит отметить, что написание кода на низкоуровневом языке, таком как x64 ассемблер, является крайней мерой для достижения максимальной производительности [69]. Это чрезвычайно трудоемкий и хрупкий процесс, который подходит только для самых критичных узлов (например, базовое матричное умножение). Для большинства случаев современные компиляторы, такие как AOCC, при правильной настройке способны генерировать код, сравнимый по производительности с ручной оптимизацией, и это гораздо более масштабируемый подход.

В заключение, стратегия компиляции для Reto-Origen должна быть основана на философии "оптимизация от начала до конца". Все компоненты фреймворка, от ядра до библиотек пользовательских операций, должны компилироваться с помощью AOCC с набором архитектурно-специфичных флагов. Процесс сборки должен включать этапы PGO и LTO для получения кода, который не только синтаксически корректен, но и семантически оптимален для целевой платформы. Такой подход позволит Reto-Origen с самого начала иметь преимущество в производительности перед фреймворками, которые используют более универсальные и менее настроенные компиляторы.

## Интеграция и Оптимизация Численных Библиотек: Роль **AOCL** в Ускорении Вычислений

Если компилятор является "конструктором", то численные библиотеки — это "двигатель" любого фреймворка машинного обучения. Именно они содержат высокооптимизированные реализации базовых линейно-алгебраических операций (BLAS), разреженной линейной алгебры (Sparse), преобразования Фурье (FFTW) и других математических функций, которые составляют сердце большинства моделей нейронных сетей. Выбор и правильная интеграция этих библиотек имеют критическое значение для производительности на центральном процессоре. Для целевой платформы AMD Ryzen 5 3600 оптимальным выбором являются AMD Optimizing CPU Libraries (AOCL). Этот набор библиотек, специально разработанный и оптимизированный для процессоров AMD "Zen", является не просто дополнением, а фундаментальным компонентом, обеспечивающим Reto-Origen конкурентное преимущество. Этот раздел подробно рассматривает состав AOCL, его преимущества перед другими

библиотеками и стратегии интеграции для достижения максимальной производительности.

AOCL представляет собой коллекцию численных подпрограмм, оптимизированных для процессоров AMD, включая EPYC, Ryzen и Threadripper, построенных на архитектурах Zen, Zen2, Zen3 и последующих [132][166][206]. Эти библиотеки не просто написаны для AMD, они скомпилированы с использованием самого мощного инструментария AMD — AOCC, что гарантирует их глубокую интеграцию с архитектурой Zen [42] [45]. AOCL включает в себя несколько ключевых компонентов:

1. **AOCL-BLAS (Basic Linear Algebra Subprograms):** Это основа для всех вычислений в нейронных сетях. BLAS определяет стандартные операции с векторами и матрицами, такие как GEMM (общее матричное умножение), GEMV (матрично-векторное умножение) и SCAL (скалярное умножение). AOCL-BLAS обычно реализован на основе проекта BLIS, который известен своей высокой производительностью и гибкостью [163][207]. AOCL предоставляет специализированные оптимизации для разных поколений Zen. Например, для Zen4/5 есть оптимизации для DGEMM, DTRSM и других ключевых операций, а для Zen3 — для DSCALV и DDOTV [130]. Для Ryzen 5 3600 (Zen2) это означает, что AOCL-BLAS будет содержать код, специально адаптированный для 256-битных AVX2 инструкций и двух FMA-устройств на ядро.

2. **AOCL-LAPACK (Linear Algebra Package):** LAPACK строится поверх BLAS и предоставляет более сложные операции, такие как LU-разложение, сингулярное значение декомпозиция (SVD) и вычисление собственных значений [130]. AOCL-LAPACK был обновлен до спецификаций Netlib LAPACK 3.12.0 и получил оптимизации для таких операций, как LU-факторизация и SVD [130]. Поскольку многие алгоритмы обучения и постобработки моделей используют эти разложения, производительность AOCL-LAPACK напрямую влияет на общую скорость обучения.

3. **AOCL-Sparse:** Этот компонент предоставляет библиотеку для работы с разреженными матрицами. Разреженные слои встречаются в различных моделях, включая некоторые типы рекуррентных нейронных сетей и графовые нейронные сети. AOCL-Sparse обеспечивает оптимизированные реализации операций с такими матрицами [43].

4. **AOCL-FFTW:** Библиотека для вычисления дискретного преобразования Фурье, которая может быть полезна в моделях, работающих с сигналами или изображениями.

5. **AOCL-LibM:** Математическая библиотека, содержащая оптимизированные реализации базовых функций, таких как синус, косинус, экспонента и логарифм [131][163].

Интеграция AOCL в Reto-Origen является стратегической необходимостью. Хотя существуют другие популярные библиотеки, такие как Intel Math Kernel Library (MKL) и OpenBLAS, AOCL часто предлагает лучшую производительность на процессорах AMD. MKL исторически был лидером производительности, но его оптимизации были сфокусированы на архитектуре Intel. Несмотря на то, что MKL теперь поддерживает AMD, AOCL, как "первоклассное решение", имеет преимущество в виде глубокой интеграции и использования AOCC [45]. Что еще более важно, тесты на процессорах AMD показывают, что OpenBLAS может быть быстрее MKL, особенно на больших матрицах [67][209]. Однако AOCL является официально поддерживаемым и оптимизированным решением от AMD, что делает его наиболее надежным выбором для долгосрочного развития Reto-Origen.

Процесс интеграции AOCL начинается с установки самих библиотек. AOCL доступен для скачивания с сайта AMD Developer. Он поставляется в виде набора готовых бинарных файлов для различных платформ. При компиляции Reto-Origen необходимо указать компилятору и линкеру, где находятся заголовочные файлы и библиотечные файлы AOCL. Это обычно делается с помощью флагов `-I/path/to/aocl/include` и `-L/path/to/aocl/lib`. Затем при связывании исполняемого файла Reto-Origen необходимо явно указать библиотеки, которые нужно включить, например, `-laoclgemm` (для BLAS) и `-laoclapack`.

Для дальнейшего упрощения интеграции и управления зависимостями AOCL предлагает функцию "Build-It-Yourself" (BIY) [130][165]. Эта функция позволяет разработчику выбрать один или несколько компонентов AOCL (например, BLAS, LAPACK, LibM) и собрать их в единый унифицированный бинарный файл (например, `libaocl.so` на Linux) [165]. Это устраняет проблемы, связанные с порядком ссылок на библиотеки и предотвращает дублирование API [130]. Для использования BIY требуется наличие Git, Python, CMake, GCC/g++ и, что самое главное, компилятора AOCC [165]. Это дает разработчикам полный контроль над

тем, какие именно библиотеки и с какими параметрами (например, модель данных LP64 или ILP64) будут включены в финальный продукт.

| Компонент AOCL | Назначение | Преимущества для **Reto-Origen** на **Ryzen 5 3600** |
|---|---|---|
| **AOCL-BLAS** | Операции линейной алгебры (GEMM, GEMV и др.). | Ядро производительности. Использует 256-битные AVX2 инструкции и 2 FMA-устройства на ядро Zen 2 для максимальной скорости матричных вычислений [130]. |
| **AOCL-LAPACK** | Расширенные операции линейной алгебры (LU, SVD, EVD). | Ускоряет сложные математические шаги в обучении и постобработке моделей, которые используются в PyTorch [130]. |
| **AOCL-Sparse** | Операции с разреженными матрицами. | Позволяет эффективно обрабатывать модели, содержащие разреженные слои, такие как графовые нейронные сети [43]. |
| **AOCL-FFTW** | Преобразование Фурье. | Полезно для моделей, работающих с сигналами и изображениями. |
| **AOCL-LibM** | Базовые математические функции (sin, exp, log). | Ускоряет вычисления в функциях активации и других математических операциях [131]. |
| **AOCL-RNG** | Генераторы псевдослучайных чисел. | Ускоряет операции, требующие случайных чисел, например, инициализация весов или dropout [163]. |

Важно понимать, что производительность AOCL сильно зависит от компилятора, с которым он был собран. Использование AOCC для компиляции самого Reto-Origen и AOCL для сборки его компонентов создает синергетический эффект, в котором каждый слой оптимизирован для следующего. Это отличает Reto-Origen от фреймворков, которые пытаются использовать универсальные библиотеки, не зная об уникальных особенностях целевого железа.

В заключение, интеграция AOCL — это не просто техническая задача, а ключевая стратегическая инвестиция в производительность Reto-Origen. Она позволяет фреймворку получить доступ к наиболее быстрым реализациям фундаментальных математических операций, специально написанным и скомпилированным для архитектуры Zen 2. Это устраняет одно из главных препятствий на пути к превосходству над PyTorch, чья производительность на CPU часто страдает из-за использования более универсальных и менее оптимизированных библиотек. Создав фреймворк, который с самого начала построен на прочном фундаменте AOCL, мы закладываем основу для достижения уровня производительности, который невозможно достичь с помощью простых программных изменений поверх существующей экосистемы.

# Управление Памятью и Параллелизмом: Ключевые Технологии для Эффективной Работы с 16 ГБ ОЗУ

Работа с системой, ограниченной 16 ГБ оперативной памяти, является самой серьезной инженерной задачей при создании Reto-Origen. Эта ограниченность напрямую противоречит тенденции к увеличению размеров моделей машинного обучения. Поэтому фреймворк должен быть спроектирован не просто как набор вычислительных операций, а как комплексная система управления ресурсами, где каждый аспект — от выделения памяти до распределения потоков — тщательно продуман для минимизации потребления памяти и максимальной эффективности использования процессорного времени. Этот раздел посвящен ключевым технологиям управления памятью и параллелизмом, которые необходимо внедрить в Reto-Origen для обеспечения его жизнеспособности и высокой производительности на заданной аппаратной конфигурации.

Первым и самым важным шагом является замена стандартного менеджера памяти операционной системы. По умолчанию Python и многие C/C++ приложения используют `ptmalloc` (часть glibc). Однако для высокоинтенсивных, коротких по времени, но многократно повторяющихся операций, характерных для обучения нейронных сетей, этот менеджер памяти может стать узким местом. Он страдает от фрагментации памяти и медленных операций выделения и освобождения [14]. Для Reto-Origen необходимо использовать специализированные, высокопроизводительные менеджеры памяти, такие как `jemalloc` или `TCMalloc`. Эти библиотеки были разработаны с учетом нужд серверных приложений и систем с высокой нагрузкой. Они используют более сложные алгоритмы выделения памяти, которые значительно снижают фрагментацию и ускоряют операции. Самый простой способ интегрировать их в Reto-Origen — это использовать переменную окружения `LD_PRELOAD` для загрузки библиотеки-заместителя перед запуском основного процесса [12] [14]. Например, можно запустить фреймворк командой `LD_PRELOAD=/path/to/libjemalloc.so python train.py`. Бенчмарки показывают, что замена стандартного выделителя памяти на `TCMalloc` может дать прирост производительности до 35% в задачах инференса [13].

Вторым критическим аспектом является управление параллелизмом и взаимодействием с архитектурой процессора. Ryzen 5 3600 имеет 12 потоков, но их эффективное использование требует знаний о его внутренней структуре. Хотя это односокетный процессор, у него есть два Core Complex Dies (CCD),

каждый со своим 16 МБ L3 кэша [9]. Это создает NUMA-подобную структуру внутри одного сокета. Reto-Origen должен осознавать эту структуру. При планировании потоков вычислений следует стремиться к тому, чтобы потоки, работающие вместе, были привязаны к ядрам, принадлежащим одному CCD, чтобы максимизировать совместное использование L3 кэша. Это можно реализовать на уровне операционной системы с помощью утилиты `numactl` или напрямую в коде фреймворка. Например, можно запустить основной процесс обучения на ядрах одного CCD, а вспомогательные процессы, такие как загрузка данных, — на ядрах другого.

Центральной техникой для управления параллелизмом является привязка к ЦП (CPU affinity) — принудительное привязывание процессов или потоков к определенным физическим ядрам процессора. Это помогает избежать дорогостоящих переключений контекста между ядрами и минимизировать потерю данных в кэше, когда поток перемещается с одного ядра на другое. Для Reto-Origen необходимо реализовать механизм, который автоматически привязывает свои вычислительные потоки к физическим ядрам, а не к логическим потокам (гиперпотокам). Исследования показывают, что для вычислительно-интенсивных задач гиперпоточность часто вредит производительности, поскольку два потока, делящие одно ядро, конкурируют за исполнительные блоки, что приводит к дополнительным накладным расходам на декодирование инструкций [14]. Поэтому оптимальной стратегией будет использование 12 физических ядер как 12 отдельных вычислительных устройств. Это можно контролировать с помощью переменных окружения OpenMP, таких как `GOMP_CPU_AFFINITY` или `KMP_AFFINITY` (если используется OpenMP от Intel) [12].

Выбор реализации OpenMP также имеет значение. По умолчанию PyTorch на Linux использует GNU OpenMP (`libgomp`) [12]. Однако на платформах AMD может быть выгодно использовать реализацию OpenMP от AOCC (`libomp.so`), которая может быть более производительной и лучше оптимизирована для архитектуры Zen [163]. Reto-Origen должен быть скомпилирован с флагом `-fopenmp` и, возможно, связан с `libomp.so` для достижения наилучших результатов.

Третий, и самый важный, аспект управления ресурсами — это работа с 16 ГБ памяти. Здесь на первый план выходят техники сжатия и оптимизации представления данных, в первую очередь квантование. Квантование — это процесс снижения числовой точности представления весов и активаций модели. Вместо 32-битных чисел с плавающей запятой (FP32) можно

использовать 16-битные (FP16/BF16) или 8-битные целые числа (INT8). Это позволяет:

- Уменьшить требования к памяти: Модель с INT8 весами занимает в 4 раза меньше места, чем модель с FP32 весами.
- Увеличить пропускную способность памяти: Можно передавать вдвое больше данных за один такт памяти при переходе от FP32 к FP16.
- Ускорить вычисления: Многие современные CPU, включая Zen 2, имеют аппаратную поддержку для быстрых 8-битных целочисленных вычислений.

Reto-Origen должен иметь встроенную, гибкую систему квантования. Она должна поддерживать различные режимы, такие как статическое, динамическое и пост-квантование весов, как это реализовано в PyTorch [110][112]. Кроме того, фреймворк должен быть совместим с инструментарием AMD Quark, который является кросс-платформенным инструментом для квантования моделей, поддерживая PyTorch и ONNX [65][66]. Процесс квантования обычно включает в себя три шага: загрузка модели, подготовка набора для калибровки и применение конфигурации квантования [107][114].

Помимо квантования, Reto-Origen должен поддерживать другие техники экономии памяти. Одной из них является **gradient checkpointing**. Во время обратного прохода для вычисления градиентов необходимо хранить промежуточные значения (активации) всех слоев. Для очень глубоких моделей это может потребовать огромного объема памяти. Gradient checkpointing позволяет отказаться от хранения большинства этих активаций и пересчитывать их по мере необходимости во время обратного прохода. Это является компромиссом между скоростью (обучение займет больше времени) и потреблением памяти (можно обучать значительно более крупные модели) [12].

| Технология | Описание | Импликации для **Reto-Origen** |
|---|---|---|
| Специализированный **Allocator** | Использование `jemalloc` или `TCMalloc` вместо стандартного `ptmalloc`. | Снижение фрагментации памяти и ускорение выделения/освобождения. Может дать прирост до 35% [13]. |
| **NUMA-Awareness** | Осознание и использование внутренней NUMA-структуры процессора (два CCD). | Максимизация использования 32 МБ L3 кэша путем привязки потоков к ядрам одного CCX [9]. |
| **CPU Affinity** | Принудительная привязка потоков к физическим ядрам, а не к гиперпотокам. | Минимизация накладных расходов на переключение контекста и улучшение кэширования [14]. |
| Оптимизация **OpenMP** | Использование производительной реализации OpenMP (например, от AOCC). | Повышение эффективности параллельных вычислений [12][163]. |
| Квантование | Снижение точности весов и активаций (FP32 -> FP16/INT8). | Уменьшение потребления памяти в 2-4 раза и потенциальное ускорение вычислений [106][107]. |
| **Gradient Checkpointing** | Пересчет активаций вместо их хранения во время обратного прохода. | Значительная экономия памяти, позволяющая обучать более крупные модели [12]. |

В заключение, управление ресурсами является не менее важным, чем сама арифметика вычислений. Reto-Origen должен быть спроектирован как "экономный хирург", который бережно относится к каждому байту памяти и каждому такту процессора. Это требует интеграции передовых технологий управления памятью, таких как `jemalloc`, и внедрения стратегий, которые учитывают архитектуру процессора, включая NUMA-сознательность и привязку к ЦП. Но самое главное — это создание мощной и гибкой системы квантования, которая позволит фреймворку справляться с ограничениями в 16 ГБ ОЗУ, делая обучение моделей, которые были бы невозможны в рамках менее оптимизированных систем.

## Архитектурные Улучшения Фреймворка: От **JIT**-Компилятора до Алгоритмических Оптимизаций

Помимо фундаментальных оптимизаций на уровне компилятора и библиотек, для достижения превосходства над PyTorch Reto-Origen должен быть построен на принципиально новой архитектуре, которая с самого начала была ориентирована на максимальную производительность на CPU. Вместо того чтобы эволюционировать из прототипа, ориентированного на GPU, Reto-Origen должен представлять собой декларативный подход к оптимизации. Этот раздел рассматривает ключевые архитектурные изменения, которые могут

кардинально повысить производительность: создание JIT-компилятора, реализация слияния операций, оптимизация макета тензоров и интеграция других продвинутых алгоритмических техник.

Ключевой архитектурной идеей для Reto-Origen должно стать создание собственного JIT-компилятора. Современные фреймворки, такие как PyTorch, после выпуска версии 2.0, начали использовать компилятор TorchInductor, который принимает на вход граф вычислений (построенный во время прогонов вперед с использованием `torch.compile`), применяет к нему ряд оптимизаций и генерирует высокоэффективный код для выполнения [12]. Reto-Origen должен развить эту идею дальше. Вместо того чтобы полагаться на существующие компиляторы, Reto-Origen должен иметь собственный компилятор, который работает с собственным промежуточным представлением (IR). Этот IR должен быть достаточно абстрактным, чтобы легко выражать различные типы операций, но и достаточно низкоуровневым, чтобы позволять глубокие оптимизации. Такой подход дает несколько преимуществ: 1. Глубокая интеграция с аппаратным обеспечением: Компилятор Reto-Origen сможет напрямую использовать знания о кэш-архитектуре Zen 2 и наборе инструкций AVX2 для генерации максимально специфичного машинного кода. 2. Продвинутые оптимизации: Компилятор сможет выполнять оптимизации, которые недоступны в более универсальных системах. Например, он может анализировать граф вычислений на предмет циклических зависимостей и применять более сложные формы пересчета. 3. Гибкость и контроль: Создание собственного IR и компилятора дает полный контроль над всем процессом от модели до выполнения, что невозможно при работе поверх существующих фреймворков.

Одной из самых мощных оптимизаций, которую может выполнять JIT-компилятор, является слияние операций. В нейронной сети последовательность операций, таких как `Conv2d` -> `BatchNorm` -> `ReLU`, на практике выполняется как три отдельных вызова ядра. Каждый вызов ядра означает обращение к памяти (чтение входных данных, запись выходных данных). Слияние операций заключается в создании одного нового, более сложного ядра, которое выполняет все три операции за один проход по данным. Это дает два основных преимущества: уменьшается количество обращений к памяти и уменьшается накладная стоимость вызова ядра. PyTorch 2.0 с TorchInductor уже делает это для многих комбинаций операций [12]. Reto-Origen должен делать это еще эффективнее, возможно, на уровне своего IR, позволяя объединять более сложные графы операций. Для операций, которые не могут быть напрямую

объединены, компилятор может применять постоянное вычисление, то есть вычислять выражения с постоянными входами во время компиляции, а не во время выполнения.

Другой важной архитектурной оптимизацией является управление раскладкой тензоров. В PyTorch по умолчанию используется раскладка `channels_first` (NCHW), где данные организованы в порядке: батч, каналы, высота, ширина. Однако для многих моделей, особенно с большим количеством сверточных слоев, гораздо эффективнее использовать раскладку `channels_last` (NHWC), где каналы идут последними. Переход на NHWC может значительно улучшить локальность кэша, потому что при обходе данных по строкам (что часто происходит в сверточных ядрах) пиксели из разных каналов, которые должны обрабатываться вместе, располагаются близко друг к другу в памяти. Бенчмарки показывают, что для Vision Transformer (ViT) моделей переход на NHWC может ускорить выполнение на 25% [13]. Reto-Origen должен по умолчанию поддерживать обе раскладки и иметь механизмы для их эффективного преобразования, а также предоставлять пользователю возможность явно выбирать наиболее подходящую раскладку для его модели.

На уровне алгоритмов фреймворк должен предлагать инструменты для реализации компромисса между памятью и временем. Как уже упоминалось, **gradient checkpointing** является ключевой техникой для обучения глубоких моделей в ограниченной памяти [12]. Reto-Origen должен предоставлять удобный API для его применения, возможно, позволяя пользователю указывать, после каких слоев сохранять активации.

Кроме того, фреймворк должен быть оптимизирован для асинхронной загрузки данных. Обучение на CPU часто ограничивается скоростью, с которой данные могут быть поданы в модель. Чтобы скрыть эту задержку, Reto-Origen должен использовать механизм, аналогичный `DataLoader` в PyTorch, который может загружать данные в отдельных процессах или потоках, пока основной процесс выполняет вычисления. Это позволяет перекрыть время ожидания ввода-вывода с вычислительным временем. Важно правильно настроить этот механизм, чтобы избежать конфликтов с основным процессом обучения, например, путем привязки worker'ов к определенным ядрам [12] [14].

Наконец, Reto-Origen должен избегать ненужных вычислений и операций. Например, в последовательностях слоев, где после `Conv2d` идет `BatchNormalization`, смещение (bias) в самом `Conv2d` является избыточным,

так как `BatchNormalization` вычитает среднее значение, что эквивалентно добавлению смещения [12]. Reto-Origen должен иметь механизм автоматического оптимизирования графа, который удаляет такие избыточные операции. Также, во время валидации или вывода, когда градиенты не нужны, фреймворк должен автоматически отключать их вычисление и хранение, что можно сделать с помощью контекстного менеджера, аналогичного `torch.no_grad()` [12].

| Архитектурная Оптимизация | Описание | Преимущества для **Reto-Origen** |
|---|---|---|
| **JIT**-Компилятор | Собственный компилятор, преобразующий IR в оптимизированный машинный код. | Глубокая интеграция с аппаратным обеспечением, продвинутые оптимизации, полный контроль над процессом выполнения. |
| Слияние операций | Объединение нескольких последовательных операций в одно вычислительное ядро. | Уменьшение обращений к памяти и накладных расходов на вызов ядра. Ускорение выполнения [12]. |
| Оптимальный макет тензоров | Поддержка и переключение между `NCHW` и `NHWC` раскладками. | Улучшение локальности кэша для сверточных и трансформерных моделей, потенциальный прирост производительности до 25% [13]. |
| **Gradient Checkpointing** | Пересчет активаций вместо их хранения для экономии памяти. | Позволяет обучать значительно более глубокие и крупные модели в условиях нехватки памяти (16 ГБ) [12]. |
| Асинхронная загрузка данных | Загрузка данных в отдельных потоках/процессах для перекрытия задержек ввода-вывода. | Предотвращение простоя вычислительных ядер из-за ожидания данных. |
| Автоматическое слияние и оптимизация | Автоматическое удаление избыточных операций (например, bias при BatchNorm) и использование `no_grad` в инференсе. | Уменьшение количества выполняемых операций и потребления памяти без вмешательства пользователя. |

В заключение, архитектурные улучшения являются тем, что отличает Reto-Origen от простого форка PyTorch. Переход к собственному JIT-компилятору и IR создает фундамент, на котором можно строить любые новые оптимизации. Реализация слияния операций, поддержка разных макетов тензоров и продвинутые техники управления памятью, такие как контроль градиентов, превращают фреймворк из универсального инструмента в специализированную машину для высокопроизводительных вычислений на CPU. Эти архитектурные решения, наряду с низкоуровневыми оптимизациями, в совокупности позволят Reto-Origen достичь уровня производительности, который будет сложно достичь конкурентам, чья архитектура не была изначально спроектирована с такой целью.

# Синтез и Дорожная Карта: Создание Нового Стандарта Производительности на CPU

Настоящее исследование было направлено на разработку комплексной стратегии для создания фреймворка Reto-Origen, который бы не просто конкурировал, а превосходил существующие решения, в частности PyTorch, по всем ключевым параметрам производительности при обучении моделей машинного обучения исключительно на процессоре AMD Ryzen 5 3600 с 16 ГБ оперативной памяти. Анализ целевой аппаратной платформы, существующих конкурентов и передовых инструментов оптимизации позволил сформировать четкую дорожную карту, основанную на принципах глубокой интеграции и специализации. Успех Reto-Origen зависит не от имитации функциональности PyTorch, а от создания нового, системно-уровневого программного обеспечения, которое рассматривает аппаратное обеспечение как центральный элемент своей архитектуры.

Ключевым выводом исследования является то, что для достижения заявленной цели Reto-Origen должен быть создан с нуля как единая, нативно-оптимизированная система, а не как порт существующего фреймворка. Его архитектура должна быть построена вокруг трех основных столпов: специализированного компилятора, системно-уровневых библиотек и продвинутого управления ресурсами. Каждый из этих столпов должен быть глубоко интегрирован с другими, создавая синергетический эффект, который в совокупности обеспечивает значительное преимущество в производительности.

Дорожная карта реализации **Reto-Origen** может быть сведена к четырем последовательным, но взаимосвязанным этапам:

Этап **1:** Создание Фундаментальной Оптимизированной Платформы. Это самый основополагающий этап, на котором закладывается вся производительность фреймворка. 1. Компиляция с **AOCC:** Все компоненты Reto-Origen, включая ядро, библиотеки операций и демонстрационные приложения, должны компилироваться исключительно с помощью AMD Optimizing C/C++ Compiler (AOCC). 2. Архитектурно-специфичные флаги: Процесс компиляции должен включать обязательные флаги, такие как `-march=znver2` для использования всех инструкций Zen 2, `-Ofast` для агрессивных оптимизаций, и `-flto=thin` для межфайловых оптимизаций [10] [42]. 3. Интеграция **AOCL:** Reto-Origen должен быть скомпилирован и связан с AMD Optimizing CPU Libraries (AOCL), в первую очередь с AOCL-BLAS и AOCL-LAPACK. Это обеспечит использование самых

быстрых реализаций линейно-алгебраических операций, оптимизированных для архитектуры Zen [45] [130]. 4. Профилирование на основе использования **(PGO):** Процесс сборки должен включать этапы генерации профилей на репрезентативных рабочих нагрузках и последующей компиляции с использованием этих профилей. Это позволит получить прирост производительности до 25% за счет оптимизации ветвлений и расположения кода [10].

Этап **2:** Разработка Системного Уровня Управления Ресурсами. На этом этапе фокус смещается с генерации кода на его выполнение в среде Ryzen 5 3600. 1. Специализированный менеджер памяти: Вместо стандартного выделителя памяти необходимо интегрировать `jemalloc` или `TCMalloc` через `LD_PRELOAD`. Это критически важно для минимизации фрагментации и ускорения операций выделения памяти, которые являются частыми в процессе обучения [13] [14]. 2. **NUMA-**сознательность и привязка к ЦП: Reto-Origen должен содержать встроенный механизм для распознавания внутренней структуры процессора (два CCD) и распределения вычислений и данных между ними. Не менее важно реализовать привязку вычислительных потоков к физическим ядрам, а не к гиперпотокам, чтобы избежать конфликтов и улучшить кэширование [14]. 3. Оптимизация **OpenMP:** Фреймворк должен быть настроен на использование наиболее производительной реализации OpenMP для AMD, например, `libomp.so` от AOCC, и правильно настроенными переменными окружения для управления привязкой потоков [12] [163].

Этап **3:** Создание Архитектуры, Оптимизированной для Алгоритмов. На этом этапе создаются высокоуровневые компоненты фреймворка, которые напрямую влияют на эффективность вычислений. 1. Разработка **JIT-**компилятора: Reto-Origen должен иметь собственный JIT-компилятор, который работает с собственным промежуточным представлением (IR). Этот компилятор будет выполнять слияние операций, оптимизацию макета тензоров (NCHW/NHWC) и другие глубокие оптимизации, недоступные в интерпретируемых системах [12]. 2. Встроенная поддержка квантования: Фреймворк должен иметь мощную и гибкую систему квантования, которая позволяет снизить точность весов и активаций до FP16 или INT8. Это ключевая технология для работы с 16 ГБ ОЗУ. Интеграция с инструментарием AMD Quark сделает эту функциональность еще более мощной [66] [107]. 3. **Gradient Checkpointing** и асинхронная загрузка данных: Эти инструменты являются обязательными для обучения моделей, размер которых превышает доступную память. Gradient checkpointing жертвует временем

ради памяти, а асинхронная загрузка данных скрывает задержки ввода-вывода [12].

Этап **4:** Стандартизированный Бенчмаркинг и Измерение Производительности. Для объективной оценки прогресса и сравнения с конкурентами необходимы стандартизированные метрики. 1. Использование **TorchBench:** Reto-Origen должен регулярно тестироваться на наборе бенчмарков TorchBench, который содержит популярные модели из мира PyTorch [10][4][18][2]. Это позволит проводить прямое сравнение производительности. 2. Использование **MLPerf:** Для более широкого сравнения с другими фреймворками и системами можно использовать модели из MLPerf Inference, таких как BERT, ResNet, ViT и др. [8]. 3. Отслеживание метрик: Необходимо отслеживать не только время обучения/вывода, но и потребление памяти, энергопотребление (если возможно) и стабильность выполнения на целевой платформе [10][3].

Риски и Ограничения: Реализация такого проекта является колоссальной инженерной задачей, сопоставимой по масштабу с созданием нового ядра операционной системы. Она требует команды высококвалифицированных системных архитекторов, специалистов по компиляторам и низкоуровневых программистов. Основным риском остается работа с 16 ГБ ОЗУ. Без продвинутой поддержки квантования и других техник экономии памяти многие современные модели попросту не смогут обучаться. Reto-Origen должен четко документировать свои ограничения по размеру моделей. Еще одним ограничением является его высокая степень специализации. Фреймворк, созданный с таким упором на архитектуру Zen 2, будет плохо работать на процессорах других архитектур (Intel, ARM, старые AMD). Это является необходимой жертвой ради достижения максимальной производительности на заданной платформе.

В конечном счете, запрос на создание фреймворка, который "уничтожит PyTorch", является сигналом к необходимости перехода от парадигмы универсальности к парадигме мастерской специализации. Reto-Origen должен стать не просто еще одним фреймворком, а системно-уровневым оптимизатором, который на границе программного обеспечения и аппаратного обеспечения достигает производительности, недостижимой для более универсальных систем. Успех этого проекта будет зависеть от того, насколько глубоко его разработчики готовы интегрироваться со всей стеком технологий, предназначенной для AMD Ryzen, и насколько смело они пойдут на

радикальные архитектурные изменения, направленные исключительно на одну цель — раскрыть весь потенциал процессора AMD Ryzen 5 3600.

---

## Справка


1. From 2s to 600ms: PyTorch vs ONNX Runtime | by Naoki Goto - Medium https://medium.com/@naokig/from-2s-to-600ms-pytorch-vs-onnx-runtime-5fb2ef14e4f0
2. Performance Discrepancy Between PyTorch and ONNX Model ... - Reddit https://www.reddit.com/r/computervision/comments/1en829d/performance_discrepancy_between_pytorch_and_onnx/
3. Does ONNX increase inference efficiency compared to pytorch ... https://discuss.pytorch.org/t/does-onnx-increase-inference-efficiency-compared-to-pytorch-model/194980
4. AMD Ryzen 5 3600 Specs - CPU Database - TechPowerUp https://www.techpowerup.com/cpu-specs/ryzen-5-3600.c2132
5. AMD Ryzen 5 3600 6-Core Benchmarks, Linux Performance https://openbenchmarking.org/s/AMD+Ryzen+5+3600+6-Core
6. MLCommons MLPerf Training Benchmark https://mlcommons.org/benchmarks/training/
7. MLPerf Training v6.0 Results: New MoE Benchmarks and ... https://mlcommons.org/2026/06/mlperf-training-v6-0-results/
8. Reference implementations of MLPerf® inference ... https://github.com/mlcommons/inference
9. Zen 2 - Wikipedia https://en.wikipedia.org/wiki/Zen_2
10. AMD Zen2 Optimization Guide (Discoverer CPU partition) https://docs.discoverer.bg/zen2.html
11. AMD Zen 2 Architecture Explored: What Makes Ryzen 3000 So ... https://hothardware.com/reviews/amd-zen-2-architecture-explained
12. Performance Tuning Guide - PyTorch documentation https://docs.pytorch.org/tutorials/recipes/recipes/tuning_guide.html
13. Optimizing PyTorch Model Inference on CPU | by Chaim Rand https://chaimrand.medium.com/optimizing-pytorch-model-inference-on-cpu-ccd3aa5884ad

14. CPU Affinity for PyG Workloads — pytorch_geometric documentation https://pytorch-geometric.readthedocs.io/en/2.6.1/advanced/cpu_affinity.html
15. Benchmarking OpenVINO™: Accelerating ONNX Inference on CPU ... https://www.linkedin.com/pulse/benchmarking-openvino-accelerating-onnx-inference-jerome-revillard-gxhie
16. chsasank/device-benchmarks - GitHub https://github.com/chsasank/device-benchmarks
17. TensorFlow vs. PyTorch vs. JAX vs. ONNX: Which Framework Powers ... https://medium.com/@thiksigar/tensorflow-vs-pytorch-vs-jax-vs-onnx-which-framework-powers-your-machine-learning-journey-55e4a7c41ec9
18. Top 7 Machine Learning Frameworks Compared | - Datafloq https://datafloq.com/top-7-machine-learning-frameworks-compared/
19. JAX vs PyTorch: Comparing Two Powerhouses in ML Frameworks https://dev.to/get_pieces/jax-vs-pytorch-comparing-two-powerhouses-in-ml-frameworks-70g
20. Benchmark of the newly launched PYTORCH 2.0 - Reddit https://www.reddit.com/r/learnmachinelearning/comments/zd6krn/benchmark_of_the_newly_launched_pytorch_20/
21. JAX vs PyTorch: Hands-On Comparison (Same Model Built ... https://www.youtube.com/watch?v=1Yzv32BGCSY
22. ML Engineer comparison of Pytorch, TensorFlow, JAX, and Flax https://softwaremill.com/ml-engineer-comparison-of-pytorch-tensorflow-jax-and-flax/
23. Comparing PyTorch and JAX - DigitalOcean https://www.digitalocean.com/community/tutorials/pytorch-vs-jax
24. Quantifying The AVX-512 Performance Impact With AMD ... https://www.reddit.com/r/hardware/comments/1ethe1e/quantifying_the_avx512_performance_impact_with/
25. Zen 5's AVX-512 Frequency Behavior - by Chester Lam https://chipsandcheese.com/p/zen-5s-avx-512-frequency-behavior
26. Quantifying The AVX-512 Performance Impact With AMD ... https://www.phoronix.com/forums/forum/hardware/processors-memory/1485793-quantifying-the-avx-512-performance-impact-with-amd-zen-5-ryzen-9-9950x-benchmarks/page2
27. Understanding AVX-512 & Validating Usage on AMD EPYC https://www.amd.com/en/blogs/2026/understanding-avx-512---validating-usage-on-amd-epyc-.html
28. AVX-512: First Impressions on Performance and ... https://shihab-shahriar.github.io/blog/2026/AVX-512-First-Impressions-on-Performance-and-Programmability/
29. AMD's Zen 5 AVX-512 performance tested https://www.tomshardware.com/pc-components/cpus/amds-zen-5-avx-512-performance-tested-zen-5-performs-significantly-better-than-zen-4-on-linux-without-consuming-any-more-power

30. AVX512 in a single cycle vs 2 cycles is big if the clock ... https://news.ycombinator.com/item?id=40559793
31. Ryzen 5 3600 Userbenchmark Result found : r/Amd https://www.reddit.com/r/Amd/comments/burpd1/ryzen_5_3600_userbenchmark_result_found/
32. Measure and accelerate PyTorch Inference on Arm servers https://learn.arm.com/learning-paths/servers-and-cloud-computing/torchbench/pytorch-benchmark/
33. Ryzen 5 3600 Full Review & Benchmarks Leak https://www.youtube.com/watch?v=pNR1iUS0Hi8
34. ROCm PyTorch Inference Benchmark https://www.ccs.tsukuba.ac.jp/wp-content/uploads/sites/14/2025/09/03.-Inference-Benchmark-Tutorial.pdf
35. Baffling performance issue on most NVidia GPUs with ... https://discuss.huggingface.co/t/baffling-performance-issue-on-most-nvidia-gpus-with-simple-transformers-pytorch-code/39292
36. PyTorch 2.8 Released With Better Intel CPU Performance ... https://www.phoronix.com/news/PyTorch-2.8-Released
37. AMD Optimizing C/C++ and Fortran Compilers (AOCC) https://www.amd.com/en/developer/aocc.html
38. AMD Optimized Compiler Collection (AOCC) - Discoverer HPC Docs https://docs.discoverer.bg/aocc.html
39. AMD Optimizing Compilers (AOCC) :: Documentation for HPC https://docs.hpc.gwdg.de/software_stacks/compilers_interpreters/amd_optimizing_compilers/index.html
40. AMD Optimizing C/C++ Compiler 2.0 Released With Zen 2 Support https://www.phoronix.com/news/AMD-AOCC-2.0-Released
41. AMD Optimizing C/C++ Compiler (AOCC) - Reddit https://www.reddit.com/r/Amd/comments/6be5qf/amd_optimizing_cc_compiler_aocc/
42. AMD AOCC Compiler - HECC Knowledge Base https://www.nas.nasa.gov/hecc/support/kb/amd-aocc-compiler_691.html
43. AMD AOCC incl. AOCL - BayernCollab https://collab.dvb.bayern/spaces/UniARZHPCKB/pages/392035549/AMD+AOCC+incl.+AOCL
44. [PDF] Best Practice Guide - AMD EPYC - PRACE https://prace-ri.eu/wp-content/uploads/Best-Practice-Guide_AMD.pdf
45. Optimizing for AMD CPUs - MSU HPCC User Documentation https://docs.icer.msu.edu/AMD_Optimizing_CPU_Libraries_and_Compilers/
46. AMD Optimizing C/C++ Compiler Suite Version 4.0.0 Flag Descriptions https://spec.org/cpu2017/flags/aocc400-flags.2022-12-08.html

47. A curated list of awesome Machine Learning frameworks ... - GitHub https://github.com/josephmisiti/awesome-machine-learning
48. cpu-optimization · GitHub Topics https://github.com/topics/cpu-optimization?o=asc&s=stars
49. mikeroyal/Machine-Learning-Guide - GitHub https://github.com/mikeroyal/Machine-Learning-Guide
50. 9 BEST GitHub Repos for AI/ML - YouTube https://www.youtube.com/watch?v=AXUZXHIHYCM
51. A curated list of awesome open source libraries to deploy, monitor ... https://ethicalml.github.io/awesome-production-machine-learning/
52. machine-learning-algorithms · GitHub Topics https://github.com/topics/machine-learning-algorithms
53. LucaZampierin/OptML: Project for Optimization for Machine Learning https://github.com/LucaZampierin/OptML
54. Learn Machine Learning with Real Code | Full GitHub Repository ... https://www.youtube.com/watch?v=eW54K_9ULQg
55. ChristosChristofidis/awesome-deep ... https://github.com/christoschristofidis/awesome-deep-learning
56. [PDF] towards automated kernel generation - in the era of llms:asurvey - TechRxiv https://www.techrxiv.org/doi/pdf/10.36227/techrxiv.176823161.17166322
57. AMD's Open Source Deep Learning Strategy https://medium.com/intuitionmachine/amds-open-source-deep-learning-strategy-14c228be6248
58. Top 8 open source AI technologies in machine learning https://opensource.com/article/18/5/top-8-open-source-ai-technologies-machine-learning
59. Building an open source ai model : r/learnmachinelearning https://www.reddit.com/r/learnmachinelearning/comments/1c1mak9/building_an_open_source_ai_model/
60. Ai2's Recommendations to OSTP to enable open-source ... https://allenai.org/blog/ostp
61. AMD Supported Open-Source Projects and Tools https://www.amd.com/en/developer/resources/open-source-projects.html
62. Featured Open Source AI Hardware You Should Consider https://www.seeedstudio.com/blog/2019/04/11/featured-open-source-ai-hardware-you-should-consider/?srsltid=AfmBOooZlX4-24KtzJwxJB-sKBhdMc24kkoxoC2_S1c7PdIZAUNmDapj
63. Top Open-Source AI/ML Frameworks in 2026 https://www.atlantic.net/gpu-server-hosting/top-open-source-ai-ml-frameworks/

64. Open sourcing AI: intellectual property at the service of ... https://www.jipitec.eu/jipitec/article/download/356/349/1838
65. AMD Quark Model Optimization Library Now Available as Open ... https://www.amd.com/de/developer/resources/technical-articles/2025/amd-quark-model-optimization-library-now-available-as-open-sourc.html
66. AMD Quark Model Optimizer - GitHub https://github.com/amd/quark
67. Benchmark Numpy with Openblas and MKL library on AMD Ryzen 3950x ... https://shaalltime.medium.com/benchmark-numpy-with-openblas-and-mkl-library-on-amd-ryzen-3950x-cpu-96184f91057f
68. Python on AMD CPUs - NERSC Documentation https://docs.nersc.gov/development/languages/python/python-amd/
69. CPU performance optimization guide - part 4 - AMD GPUOpen https://gpuopen.com/learn/cpu-performance-guide/cpu-performance-guide-part4/
70. Inference time on CPU varying massively with training ... https://discuss.pytorch.org/t/inference-time-on-cpu-varying-massively-with-training-weight-choice/177730
71. High-Performance Llama 2 Training and Inference with ... https://pytorch.org/blog/high-performance-llama-2/
72. Lightning Talk: The Fastest Path to Production: PyTorch ... https://www.youtube.com/watch?v=HkyWFIbs4JY
73. Ray Train Benchmarks — Ray 2.56.0 - Ray Docs https://docs.ray.io/en/latest/train/benchmarks.html
74. Ryzen 5 3600: CPU has 25% less performance than host ... https://www.reddit.com/r/VFIO/comments/1qpm4ox/ryzen_5_3600_cpu_has_25_less_performance_than/
75. PyTorch 2 GPU Performance Benchmarks (Update) - aime.info https://www.aime.info/blog/en/pytorch-2-gpu-performace-benchmark-comparison/
76. GitHub - HorizonRobotics/alf: Agent Learning Framework https://alf ... https://github.com/HorizonRobotics/alf
77. GitHub - ICTMCG/Awesome-Machine-Generated-Text https://github.com/ICTMCG/Awesome-Machine-Generated-Text
78. adamyi/awesome-stars: Awesome List of my own! - GitHub https://github.com/adamyi/awesome-stars
79. IBM/SystemML_Usage: Demonstrate how to perform a ... - GitHub https://github.com/IBM/SystemML_Usage
80. shaas1704/Text2Voice: Open source machine learning ... - GitHub https://github.com/shaas1704/Text2Voice
81. NixOS-Guide/README.md at main - GitHub https://github.com/mikeroyal/NixOS-Guide/blob/main/README.md

82. Experiments with reasoning models, training techniques, papers https://github.com/benjaminzwhite/reasoning-models
83. Argus-Framework/argus-ai-debate: Multi-agent AI ... - GitHub https://github.com/Argus-Framework/argus-ai-debate
84. First Principles Framework — Core Conceptual Specification https://gist.github.com/jtprogru/dbf54077d191d575ace39b6245702be8
85. dev-trackers/awesome-python-1 - GitHub https://github.com/dev-trackers/awesome-python-1
86. ONNX Runtime much slower than PyTorch (2-3x slower) #12880 https://github.com/microsoft/onnxruntime/issues/12880
87. 9x model serving performance without changing hardware : r/Python https://www.reddit.com/r/Python/comments/1gm0flj/9x_model_serving_performance_without_changing/
88. JAX vs PyTorch vs Numpy - Basic Matrix Operations on M2 CPU https://www.youtube.com/watch?v=oebjNlaRVoc
89. Onnxruntime vs PyTorch - Stack Overflow https://stackoverflow.com/questions/67943173/onnxruntime-vs-pytorch
90. Edge AI: TensorFlow Lite vs. ONNX Runtime vs. PyTorch Mobile - DZone https://dzone.com/articles/edge-ai-tensorflow-lite-vs-onnx-runtime-vs-pytorch
91. Is JAX a better choice to focus on over PyTorch now? - Reddit https://www.reddit.com/r/learnmachinelearning/comments/16vgfed/is_jax_a_better_choice_to_focus_on_over_pytorch/
92. Deep Learning with PyTorch & ONNX - YouTube https://www.youtube.com/watch?v=X2IH2F5jius
93. JAX-sklearn: JAX-Accelerated Machine Learning - GitHub https://github.com/chenxingqiang/jax-sklearn
94. Which Deep Learning Framework Should I Choose: TensorFlow ... https://discuss.huggingface.co/t/which-deep-learning-framework-should-i-choose-tensorflow-pytorch-or-jax/160611
95. Accelerate and simplify Scikit-learn model inference with ONNX ... https://opensource.microsoft.com/blog/2020/12/17/accelerate-simplify-scikit-learn-model-inference-onnx-runtime/
96. Accelerate and Productionize ML Model Inferencing Using Open ... https://odsc.medium.com/accelerate-and-productionize-ml-model-inferencing-using-open-source-tools-7856e73f8e35
97. Is anyone using scikit for NN? Why/why not? - Hacker News https://news.ycombinator.com/item?id=28523114

98. GitHub - mikeroyal/Pop_OS-Guide https://github.com/mikeroyal/Pop_OS-Guide
99. GitHub - avinashshenoy97/awesome-python-1 https://github.com/avinashshenoy97/awesome-python-1
100. Awesome-LLM: a curated list of Large Language Model - GitHub https://github.com/hannibal046/awesome-llm
101. mikeroyal/Self-Hosting-Guide - GitHub https://github.com/mikeroyal/self-hosting-guide
102. New submissions for Tue, 29 Dec 20 · Issue #247 - GitHub https://github.com/kobiso/daily-arxiv-noti/issues/247
103. LukasHedegaard/pytorch-benchmark: Easily ... https://github.com/LukasHedegaard/pytorch-benchmark
104. Standardizing CPU Benchmarking with TorchBench for PyTorch... https://www.youtube.com/watch?v=HD-Y95I_RFA
105. Benchmarks https://www.opennn.net/benchmarks/
106. Deep Dive on PyTorch Quantization - Chris Gottbrath - YouTube https://www.youtube.com/watch?v=c3MT2qV5f9w
107. AMD Quark for PyTorch https://quark.docs.amd.com/latest/pytorch/basic_usage_pytorch.html
108. API Overview — Guide to Core ML Tools - Apple https://apple.github.io/coremltools/docs-guides/source/opt-quantization-api.html
109. Releases · amd/ZenDNN-pytorch-plugin - GitHub https://github.com/amd/ZenDNN-pytorch-plugin/releases
110. Quantization — PyTorch 2.13 documentation https://docs.pytorch.org/docs/stable/quantization.html
111. A beginner's guide to deploying LLMs with AMD on Windows using ... https://gpuopen.com/learn/pytorch-windows-amd-llm-guide/
112. Quantization and performance optimization | How-to guides https://developer.meta.com/ai/docs/how-to-guides/quantization/
113. Can you share your Ryzen 5 3600 Overclock settings? : r/Amd - Reddit https://www.reddit.com/r/Amd/comments/sz0o1x/can_you_share_your_ryzen_5_3600_overclock_settings/
114. AMD Quark - vLLM https://docs.vllm.ai/en/v0.26.0/features/quantization/quark/
115. AI Benchmark Showdown: Speed & Scale of 4 ML Stacks https://medium.com/@sparknp1/ai-benchmark-showdown-speed-scale-of-4-ml-stacks-9622154c4bde
116. ONNX vs PyTorch Speed: In-Depth Performance Comparison - Dev-kit https://dev-kit.io/blog/machine-learning/onnx-vs-pytorch-speed-comparison

117. ONNX vs PyTorch: A Comprehensive Comparison - codegenes.net https://www.codegenes.net/blog/onnx-vs-pytorch/
118. Machine Learning Libraries Compared (2026) - Zaira Labs https://zairalabs.ai/guide/guides/ml-library-compared/
119. Results http://inferena.tech/
120. AVX2 brings no performance improvement? · Issue #9417 - GitHub https://github.com/deeplearning4j/deeplearning4j/issues/9417
121. pytorch-benchmark https://pypi.org/project/pytorch-benchmark/
122. GPU vs CPU benchmark https://nlp.johnsnowlabs.com/docs/en/CPUvsGPUbenchmark_healthcare
123. PyTorch Benchmark — PyTorch Tutorials 2.13.0+cu130 ... https://docs.pytorch.org/tutorials/recipes/recipes/benchmark.html
124. Catchup results for Machine Learning on Tue, 16 Jun 2026 - arXiv https://arxiv.org/catchup/cs.LG/2026-06-16?abs=True&page=1
125. liuzhenqi77/awesome-stars: A curated list of my ... - GitHub https://github.com/liuzhenqi77/awesome-stars
126. Machine Learning Jun 2025 - arXiv https://www.arxiv.org/list/cs.LG/2025-06?skip=325&show=2000
127. GitHub - ccat3z/stars: A simple tool for creating your own awesome list https://github.com/ccat3z/stars
128. Computer Science Feb 2025 - arXiv http://arxiv.org/list/cs/2025-02?skip=7300&show=2000
129. To navigate to a certain userID, you'll need to append the parameter ... https://gist.github.com/VirenMohindra/254e4b5aafdd64e8f5c4339f31944c23
130. AMD Optimizing CPU Libraries (AOCL) https://www.amd.com/en/developer/aocl.html
131. AOCL User Guide https://www.amd.com/content/dam/amd/en/documents/pdfs/developer/aocl/aocl-v4.0-ga-user-guide.pdf
132. AMD Optimizing CPU Libraries (AOCL) https://docs.archer2.ac.uk/software-libraries/aocl/
133. AMD Optimizing CPU Libraries (AOCL) https://www.nas.nasa.gov/hecc/support/kb/amd-optimizing-cpu-libraries-(aocl)_690.html
134. AOCL https://www.psc.edu/resources/software/aocl/
135. Welcome to the torchao Documentation https://pytorch.org/ao/index.html
136. [PDF] YAHPO Gym - An Efficient Multi-Objective Multi-Fidelity Benchmark ... https://arxiv.org/pdf/2109.03670

137. Decoupled DiLoCo for Resilient Distributed Pre-training - arXiv https://arxiv.org/html/2604.21428v1
138. Adversarial Query Synthesis via Bayesian Optimization - arXiv https://arxiv.org/html/2603.01570v1
139. [PDF] Ultra-Low-Cost Drug Discovery using Hyperdimensional Computing https://arxiv.org/pdf/2106.02894
140. Low Rank Learning for Offline Query Optimization - arXiv https://arxiv.org/html/2504.06399v1
141. Deep Learning-Based Beamforming Design Using Target Beam ... https://arxiv.org/html/2507.10063v2
142. [PDF] On Extending Amdahl's law to Learn Computer Performance - arXiv https://arxiv.org/pdf/2110.07822
143. RESTRuler: Towards Automatically Identifying Violations of RESTful ... https://arxiv.org/html/2402.13710v1
144. Efficient, Transparent Temporal Multiplexing for Consumer GPUs https://arxiv.org/html/2601.11743v1
145. [PDF] Hybrid Classical-Quantum Transfer Learning with Noisy ... - arXiv https://arxiv.org/pdf/2603.16973
146. Comparision Between TensorRT, Pytorch, ONNX Runtime - Medium https://medium.com/@sundarbalamurugan/comparision-between-tensorrt-pytorch-onnx-runtime-2842bd208d73
147. ViT PyTorch vs JAX training benchmarks on Vertex AI ... - GitHub https://github.com/GoogleCloudPlatform/vertex-ai-samples/blob/main/community-content/vertex_model_garden/benchmarking_reports/jax_vit_benchmarking_report.md
148. ONNX Runtime Benchmark - OpenBenchmarking.org https://openbenchmarking.org/test/pts/onnx&eval=13c3b2908d7d0186ce944119892bbb97759a0f68
149. JAX vs PyTorch: A simple transformer benchmark - Echo Nolan's Blog https://www.echonolan.net/posts/2021-09-06-JAX-vs-PyTorch-A-Transformer-Benchmark.html
150. benchmark.py - microsoft/onnxruntime - GitHub https://github.com/microsoft/onnxruntime/blob/main/onnxruntime/python/tools/transformers/benchmark.py
151. Awesome lists about all kinds of interesting topics - GitHub https://github.com/sindresorhus/awesome
152. Updated 2026-08-04 - GitHub Gist https://gist.github.com/masta-g3/8f7227397b1053b42e727bbd6abf1d2e
153. Scikit-Learn Benchmark - OpenBenchmarking.org https://openbenchmarking.org/test/pts/scikit-learn&eval=8e88070d968a829defb4a8676b80ce8b60d39bcc

154. Choosing the Right ML Framework for Your AI Deployment | Trueface https://medium.com/trueface-ai/deep-learning-framework-benchmarks-cpu-inference-speed-and-memory-usage-9cc58f5dd220
155. 5-Step Machine Learning Performance Comparison Guide (2026) https://www.chatbench.org/machine-learning-performance-comparison/
156. A Benchmark Environment for Offline Reinforcement Learning in ... https://arxiv.org/html/2407.09415v1
157. [PDF] Data Model Design and Feature Management for Emerging Energy ... https://arxiv.org/pdf/2205.04267
158. [PDF] Defending against the Label-flipping Attack in Federated Learning https://arxiv.org/pdf/2207.01982
159. [PDF] Assets Forecasting with Feature Engineering and Transformation ... https://arxiv.org/pdf/2501.07580
160. [PDF] Evaluation of Variational Quantum Classifiers (VQC) for Cyberattack ... https://arxiv.org/pdf/2606.21715
161. AMD Optimizing C/C++ Compiler 4.0 'AOCC' Makes Zen 4 ... - Reddit https://www.reddit.com/r/Amd/comments/yvub6l/amd_optimizing_cc_compiler_40_aocc_makes_zen_4/
162. [PDF] AOCC User Guide | AMD https://www.amd.com/content/dam/amd/en/documents/pdfs/developer/aocc/aocc-v4.0-ga-user-guide.pdf
163. AMD Optimizing CPU Libraries (AOCL) - Discoverer HPC Docs https://docs.discoverer.bg/aocl.html
164. AMD Optimizing C/C++ Compiler 4.0 'AOCC' Makes Zen 4 Ryzen ... https://wccftech.com/amd-optimizing-c-c-compiler-4-0-aocc-makes-zen-4-ryzen-epyc-cpus-run-faster/
165. amd/aocl - GitHub https://github.com/amd/aocl
166. AMD Optimizing CPU Libraries (AOCL) https://www.amd.com/ja/developer/aocl.html
167. OriGen: Enhancing RTL Code Generation with Code-to ... - GitHub https://github.com/pku-liang/origen
168. [2407.16237] OriGen:Enhancing RTL Code Generation with ... - arXiv https://arxiv.org/abs/2407.16237
169. From articles to code: on-demand generation of core algorithms from ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12857569/
170. [PDF] Registries in Machine Learning-Based Drug Discovery: A Shortcut to ... https://research.chalmers.se/publication/543153/file/543153_Fulltext.pdf

171. (PDF) A Machine Learning Ontology - ResearchGate https://www.researchgate.net/publication/344772818_A_MACHINE_LEARNING_ONTOLOGY
172. Automatically tracking metadata and provenance of machine ... https://www.amazon.science/publications/automatically-tracking-metadata-and-provenance-of-machine-learning-experiments
173. Towards a Retrieval-Augmented Generation Framework for ... - MDPI https://www.mdpi.com/2227-7102/15/6/706
174. RePORT RePORTER https://reporter.nih.gov/project-details/9767258
175. Research | Advancing AI, ML & Data Innovation - Factored AI https://www.factored.ai/resources/publications
176. https://dx.doi.org/10.5220/0009094204690475 https://dx.doi.org/10.5220/0009094204690475
177. Ryzen 5 3600 | 2024 Revisit [42 Game Benchmark - YouTube https://www.youtube.com/watch?v=QuxI2zaTG7I
178. Low benchmark for Ryzen 5 3600? : r/Amd - Reddit https://www.reddit.com/r/Amd/comments/ik2wrm/low_benchmark_for_ryzen_5_3600/
179. AMD Ryzen 5 3600 CPU Review & Benchmarks - Gamers Nexus http://gamersnexus.net/hwreviews/3489-amd-ryzen-5-3600-cpu-review-benchmarks-vs-intel
180. AMD Ryzen 5 3600 100-100000031BOX - CPU UserBenchmarks https://cpu.userbenchmark.com/AMD-Ryzen-5-3600/Rating/4040
181. AMD Ryzen 5 3600 Review, Price to Performance Champ! - Reddit https://www.reddit.com/r/Amd/comments/cazxpi/amd_ryzen_5_3600_review_price_to_performance_champ/
182. pytorch/benchmark: TorchBench is a collection of open ... - GitHub https://github.com/pytorch/benchmark
183. AMD Ryzen 5 3600 CPU Review & Benchmarks - YouTube https://www.youtube.com/watch?v=7AbNeht4tAE
184. [PDF] From Machine Learning Documentation to Requirements - arXiv https://arxiv.org/pdf/2511.15340
185. A Machine Learning Framework for Identifying Sources of AI ... https://iapress.org/index.php/soic/article/view/2225
186. [PDF] RE-centric Recommendations for the Development of Trustworthy(er ... https://arxiv.org/pdf/2306.01774
187. Projects - Aythami Morales https://aythami.me/research-areas/projects/
188. What is the best AMD Ryzen 5 laptops for running machine learning ... https://discuss.huggingface.co/t/what-is-the-best-amd-ryzen-5-laptops-for-running-machine-learning-models-and-libraries-like-tensorflow-or-pytorch/109264

189. Best Laptops for Data Scientists (including AI & ML) - YouTube https://www.youtube.com/watch?v=Tow21xoMhL0
190. [PDF] Performance Study on CPU-based Machine Learning with PyTorch https://par.nsf.gov/servlets/purl/10462528
191. Automated systematic reviews by using Deep Learning and ... - GitHub https://github.com/Rensvandeschoot/automated-systematic-review
192. GitHub - eriklindernoren/ML-From-Scratch: Machine Learning From ... https://github.com/eriklindernoren/ML-From-Scratch
193. lambda7xx/awesome-AI-system: paper and its code for AI ... - GitHub https://github.com/lambda7xx/awesome-AI-system
194. GitHub - dair-ai/ML-Papers-Explained: Explanation to key concepts ... https://github.com/dair-ai/ML-Papers-Explained
195. patrickmineault/ml-theory-reading-list: An annotated reference list of ... https://github.com/patrickmineault/ml-theory-reading-list
196. [PDF] Instructions for *ACL Proceedings - OpenReview https://openreview.net/pdf?id=9NWKrRoJJJ
197. GitHub - mcanini/SysML-reading-list: Systems for ML/AI & ML/AI for ... https://github.com/mcanini/SysML-reading-list
198. GitHub - eugeneyan/applied-ml: Papers & tech blogs by companies ... https://github.com/eugeneyan/applied-ml
199. [PDF] HiRace: Accurate and Fast Source-Level Race Checking of GPU ... https://arxiv.org/pdf/2401.04701
200. Valid Conformal Prediction for Dynamic GNNs - arXiv https://arxiv.org/html/2405.19230v1
201. Multitask finetuning and acceleration of chemical pretrained models ... https://arxiv.org/html/2510.12719v1
202. Sparse Sub-Quadratic Attention with Hierarchical Attention Pruning https://arxiv.org/html/2406.09827v1
203. Fast and Generalizable parameter-embedded Neural Operators for ... https://arxiv.org/html/2508.08087v1
204. Is python onnxruntime-gpu slower than pytorch cuda ? #2750 - GitHub https://github.com/microsoft/onnxruntime/issues/2750
205. Test and tune for Zen 2 · Issue #2180 · OpenMathLib/OpenBLAS https://github.com/OpenMathLib/OpenBLAS/issues/2180
206. AMD Optimizing CPU Libraries (AOCL) https://www.amd.com/en/developer/zen-software-studio/applications/aocl/spack-support.html

207. Using AMD® Optimizing CPU Libraries (AOCL) in ... - COMSOL https://www.comsol.com/support/knowledgebase/1311

208. [PDF] AMD Ryzen™ Processor Software Optimization https://gpuopen.com/download/GDC-2023-AMD-Ryzen-Processor-Software-Optimization.pdf

209. OpenBLAS is faster than Intel MKL on AMD Hardware (Ryzen) https://discourse.julialang.org/t/openblas-is-faster-than-intel-mkl-on-amd-hardware-ryzen/8033

210. Openblas benchmarks for ryzen 3000 anyone? : r/Amd - Reddit https://www.reddit.com/r/Amd/comments/caa41a/openblas_benchmarks_for_ryzen_3000_anyone/

211. What, if anything, can be done to optimize performance for modern A... https://www.mathworks.com/matlabcentral/answers/396296-what-if-anything-can-be-done-to-optimize-performance-for-modern-amd-cpu-s

212. [PDF] AOCC User Guide | AMD https://www.amd.com/content/dam/amd/en/documents/developer/version-4-1-documents/aocc/aocc-4.1-user-guide.pdf

213. Local AI benchmarks on RTX 4070, RX 580, Apple M1 ... - Facebook https://www.facebook.com/groups/marketingngrowth/posts/1014982500986503/

214. [PDF] A novel lightweight skeletal temporal model for real-time ... - SciOpen https://www.sciopen.com/local/article_pdf/10.1007/s12273-025-1335-6.pdf

215. What to Buy for Local LLMs (April 2026) | by Julien Simon | Medium https://julsimon.medium.com/what-to-buy-for-local-llms-april-2026-a4946a381a6a

216. The fallacy of 'synthetic benchmarks' : r/hardware - Reddit https://www.reddit.com/r/hardware/comments/jvq3do/the_fallacy_of_synthetic_benchmarks/

217. A Full Hardware Guide to Deep Learning - Tim Dettmers https://timdettmers.com/2018/12/16/deep-learning-hardware-guide/

218. Ensemble transformer with post-hoc explanations for depression ... https://www.sciencedirect.com/science/article/pii/S2589004225028664

219. Everyone tells you to buy a CPU with an NPU for local AI ... - Instagram https://www.instagram.com/reel/Da7vgAPic4y/

220. Making AMD GPUs competitive for LLM inference (2023) https://news.ycombinator.com/item?id=42498634

221. Geekbench, el popular benchmark multiplataforma de Primate Labs ... https://www.instagram.com/p/DbL0BkzIOwO/

222. Meet our PRO MAX Edge AI+, featuring the all-new MSI AI Solution ... https://www.instagram.com/reel/DZKA937j6JO/

223. [PDF] amd ryzen™ processor software optimization https://gpuopen.com/download/gdc-2019-s2-amd-ryzen-processor-software-optimization.pdf

224. performance on AMD Ryzen and Threadripper · Issue #1461 - GitHub https://github.com/xianyi/OpenBLAS/issues/1461

225. Need r/AMD Community Idiot's Guide To Manual Ryzen 3000 ... https://www.reddit.com/r/Amd/comments/cf0klv/need_ramd_community_idiots_guide_to_manual_ryzen/

226. The Ultimate AMD Ryzen 3000 Overclocking Guide - YouTube https://www.youtube.com/watch?v=n4NvGG_TS-M

227. How To Use MKL with AMD Ryzen and Threadripper CPU's ... https://www.pugetsystems.com/labs/hpc/how-to-use-mkl-with-amd-ryzen-and-threadripper-cpu-s-effectively-for-python-numpy-and-other-applications-1637/

228. AMD Zen 4 Software Optimization Guide | PDF | Cpu Cache - Scribd https://www.scribd.com/document/900099022/GDC-2023-AMD-Ryzen-Processor-Software-Optimization

229. Key Points to Overclock Zen 2 & 3, Ryzen 3000 & 5000 Series CPU's https://intensewebs.com/index.php/amd/amd-cpu/key-points-ryzen-oc